FGS Press

Journal of Computer Allied Intelligence
ISSN: 2584-2676
Vol.03(03), June 2025, pp.28-39

Research Article

Prometheus & Grafana: A Metrics-focused Monitoring Stack

Mohammed Daffalla Elradi[1,*]
[1]Professor, Department of Communication Systems Engineering, University of Science and Technology, Khartoum, Sudan
[*]Corresponding Author: Mohammed Daffalla Elradi. Email: mohd_daf_elradi@hotmail.com
Received: 20/05/2025; Revised: 06/06/2025; Accepted:14/06/2025; Published: 30/06/2025.
DOI: https://doi.org/10.69996/jcai.2025015

Abstract: Observability is a crucial process in the evolving IT sector and other sectors as well. Having a holistic view of your infrastructure is not an option anymore but a necessity, especially for monitoring the performance and availability of critical systems that might lead to catastrophic impacts if anything goes wrong for some reason. Having a reliable tool for that purpose remains an essential factor for a smooth operation pace. It might be challenging to choose a tool among the proliferation of tools in the market, each promising to provide the best result expected**.** In this paper, a monitoring stack composed of open-source tools (Prometheus, Node Exporter, and Grafana) was leveraged to collect, store, and visualize metrics from a Linux server in real time. Docker had been used for a rapid deployment of the services of the stack, and Node Exporter Full dashboard was imported for a quick yet informative visualization. It was found to provide detailed insights into the monitored infrastructure in a very intuitive way.

Keywords: prometheus, node exporter, grafana, dashboard, docker.

1.Introduction

Monitoring is a very crucial process in the modern IT lifecycle and is not an option anymore, especially with the increasing cyber-attacks and performance-challenging incidents. Adopting the CIA triad (confidentiality, integrity, and availability) mandates the availability of data, resources, or infrastructure to the authorized entities, which requires monitoring to ensure a reliable secure environment. Having a holistic view about current performance indicators is a game-changer for assessing past, current and predicting future systems' outcome [1].

Monitoring IT assets in real-time requires a reliable monitoring system that collects and provides intuitive information within small to extended networks to guarantee a smooth operation without disruption [2].

Having insightful data visualizations greatly influence the observability capabilities required to have a clear view about what's going on, what needs to be adjusted and what needs to be checked and enhanced. That merely relies on collecting precise metrics from end-devices in real-time or semi real-time to help efficient decision-making and avoid unavailability or downgraded performance.

Observability in IT refers to the ability to understand the internal state of a system, primarily through three key pillars: logs, metrics and traces [16],, which will be discussed below:

- Logs:

Are detailed records of events that occur within a system, capturing timestamps, event types, and other relevant data. They are essential for troubleshooting and provide granular insights into system behavior but can generate large volumes of data that might impact system performance if not managed properly.

- Metrics:

These are numerical measurements that reflect system performance, such as CPU usage, memory consumption, and error rates. They provide real-time or historical data to identify patterns and trends over time intervals to provide a baseline for performance.

- Traces:

Offers a comprehensive view of the flow of requests through a system, helping to identify bottlenecks and performance issues as well as providing insights into latency and dependencies. While traces are valuable for understanding complex interactions, they can generate significant data volumes also and may require careful sampling to manage effectively.

Figure 1 below shows the three pillars of observability.

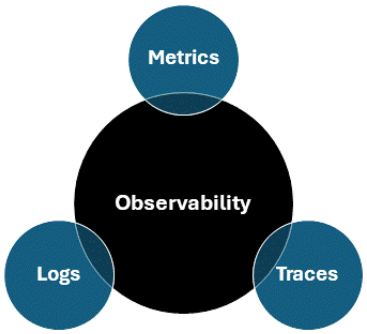

Figure 1: The three pillars of observability

It is also important to consider that the use case for monitoring differs from one industry to another, and monitoring strategies should be tailored accordingly to adapt to emerging requirements and challenges. Some examples of these sectors include banking, healthcare, telecom, and manufacturing [3].

There are some challenges that emerge for monitoring solutions, which can be summarized below:

- Fragmented monitoring tools:

The difficulty of having a unified monitoring tool is due to the existence of multiple monitoring tools for different devices and systems.

- Distributed networks:

The challenge imposed by having infrastructure spanning across multiple geo-locations makes it difficult to consolidate data for a meaningful overview.

- Expanding the scope of monitoring:

The scope of monitoring might expand beyond traditional IT-related aspects, requiring support for miscellaneous device types and protocols.

- Specialized teams and access:

Since there are different teams responsible for different services and systems ranging from networks, databases, …etc, there is a need for role-based access and specialized visualizations correspondingly. Those challenges need to be considered when choosing the appropriate monitoring tool [4].

As there is a proliferation of monitoring tools in the market, finding a tool that can provide ultimate results is a real challenge as well, depending on the use case and targeted infrastructure. For instance, the Elastic stack is ideal for centralized logging and log monitoring. This paper will focus on Prometheus and Grafana, which are suitable for performance metrics and services monitoring [5].

1.1.Prometheus

Prometheus is an open-source monitoring and alerting tool that is designed for reliability in metrics collection. Prometheus is adept at collecting and storing time series data with timestamps. The collection is conducted using a pull model over HTTP. It enables users to easily visualize and understand key performance metrics that reflect applications' health and availability. The architecture of Prometheus incorporates several components, each serving a unique function in the collection and visualization of collected metrics, which will be explained below [6].

1.1.1 Prometheus server

The main component is the Prometheus server, which is responsible for scraping various metrics from targeted infrastructure either directly or via middleware push gateway for short-lived jobs.

1.1.2 Client libraries

Client libraries play a pivotal role in the process of transmitting the current state of the monitored metrics when scraped by the Prometheus server. There are miscellaneous client libraries depending on the language in which the targeted application is written [7].

1.1.3 Push gateway

It serves as a metrics cache rather than a push-based monitoring system, as it does not aggregate data and is not designed for event storage. It allows temporary and batch jobs to send their metrics to Prometheus, as they might not last enough for direct scraping. The push gateway is intended for service-level metrics, while machine-level metrics are better handled by the Node Exporter [8].

1.1.4 Exporters

Exporters facilitate the collection of metrics from third-party systems that cannot be directly instrumented, such as HAProxy or Linux stats, ensuring the conversion of those metrics into a format that Prometheus can understand [9].

1.1.5 AlertManager

The AlertManager manages alerts from different client applications like Prometheus. It consolidates similar alerts into a single notification to reduce clutter. Inhibition prevents unnecessary alerts when there are already other alerts been triggered. Additionally, silences can be configured to suppress alerts for a specified time based on a pre-configured condition [10].

1.2.Grafana

Is an open-source tool that empowers the query, visualization, and analysis of metrics from various locations and data sources including Prometheus. The strength of Grafana is the ability to create intuitive dashboards involving time-based line, area, and bar charts. In addition to gauges, histograms, and heatmaps. Below are the different use cases of Grafana [11].

- **Visualization of time-series data**

Grafana is highly effective in creating interactive dashboards for metrics collected.

- Monitoring infrastructure and applications' performance

It facilitates the monitoring of metrics like CPU and memory usage, including response time and error rates, by integrating with tools like Prometheus.

- Log analysis

It can aggregate logs with metrics from systems like Elasticsearch and Loki.

- Alerting functions

Alert rules can be established to send notifications for critical events.

- Capacity planning

Analyzing historical metrics and providing predictions for optimal resource utilization and capacity planning.

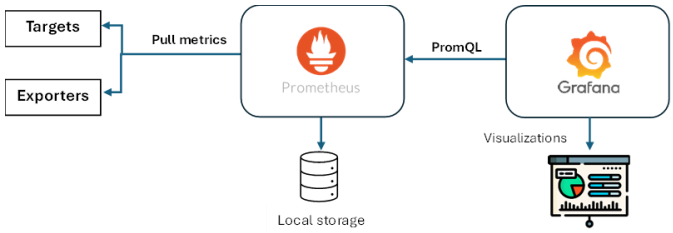

Figure 2: Prometheus and Grafana Architecture

Figure 2 demonstrates the architectural flow of Prometheus and Grafana and how they work together to monitor metrics on targets and visualize them in insightful dashboards.

- Applications/Exporters expose metrics.
- Prometheus pulls metrics based on specific scraping time.
- Metrics are stored in the local database of Prometheus in time-series format.
- Grafana queries Prometheus using PromQL (Prometheus Query Language).
- Grafana creates interactive visualizations and dashboards.

2. Literature Review

Many studies have been conducted for Prometheus and Grafana, highlighting its feasibility in monitoring applications and servers. Some will be discussed below:

In [12], an effective infrastructure monitoring solution was implemented using Prometheus, Grafana, and Node Exporter in the Kubernetes environment, mainly driven by the need for real-time monitoring, which was challenging in infrastructure monitoring. Prometheus served as the main component, employing a pull-based method to collect metrics from Kubernetes nodes and pods. Grafana was used to offer advanced visualization capabilities. Node exporter was further utilized to enrich the monitoring by providing detailed metrics at the node level. The synergy of these tools allowed for comprehensive visibility of the infrastructure and enhanced the reliability and performance of Kubernetes deployments, improving application availability and operational efficiency.

Analyzing vast amounts of data that might reach up to more than a thousand exabytes and a daily average of about 2.5 exabytes across global data centers can be challenging. Grafana enables users to query, visualize, and alert on metrics regardless of their storage location, transforming time-series formatted data into accessible graphs and visualizations. The paper also compared Grafana with other visualization tools like Power BI and Tableau [13].

The seamless operation and availability of critical applications have become paramount, as even minor performance issues might lead to substantial loss in revenue and customer satisfaction. To mitigate these risks, organizations should not rely on traditional monitoring tools as they are increasingly seen as inefficient and reactive. In contrast, modern tools like Prometheus and Grafana can be leveraged for real-time and reliable monitoring.

Prometheus serves as a robust metrics collection engine, presenting detailed data on application behavior and infrastructure health, while Grafana serves as the visualization provider platform. Effective implementation of those tools requires careful planning to adapt to business

needs. Case studies demonstrate how leading companies like Amadeus IT Group and JP Morgan Chase have successfully utilized Prometheus and Grafana to enhance their service's reliability and operational efficiency [14-17].

In [18], Prometheus and Grafana were implemented to monitor error rates and latency within microservices to maintain application health and performance in addition to the use of variables in monitoring configuration, which enhances flexibility and reusability.

This paper will discuss the implementation of Prometheus and Grafana for monitoring metrics and its effectiveness in creating real-time.

3. Method

This paper will highlight the features that make Prometheus and Grafana stand out for the use case of metrics collection due to the time-series format empowered by Prometheus,

Prometheus and Grafana had been implemented in a server with the specifications detailed in Table 1 below. The system was implemented in VMware® Workstation 17 Pro.

Table 1: Server specifications

Specification	*Details*
System	VMware Virtual Platform
Processor	Intel(R) Core (TM) i7-7500U CPU @ 2.70GHz
Hard Disk	50 GB
RAM	8 GB
Operating	Rocky Linux 9.2

The Elastic Stack was installed following the official guide on the server mentioned in Table 1. Regarding the client side, where logs are to be collected, it was implemented in a device described in Table 2.

Table 2: Client specifications

Specification	*Details*
System	HP Pavilion Notebook
Processor	Intel(R) Core (TM) i7-7500U CPU @ 2.70GHz
Hard Disk	512 GB
RAM	8 GB
Operating	Microsoft Windows 10 Pro

The process of implementing the monitoring stack composed of Prometheus and Grafana had been made using docker-compose. That included structuring a directory, as below illustrated in Figure 3. Each file will be explained.

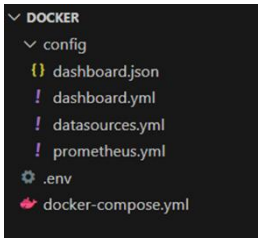

Figure 3: Directory structure

- .env file

The first step was defining an environmental variables file to ease the management of configuration across different environments centrally and avoid hardcoding sensitive information in the docker-compose file.

```
NODE_EXPORTER_CONTAINER=node-exporter
NODE_EXPORTER_PORT=9100
PROMETHEUS_CONTAINER=prometheus
PROMETHEUS_PORT=9090
GRAFANA_CONTAINER=grafana
GRAFANA_PORT=3000
```

- prometheus.yml file

Includes the configuration of Prometheus, where the scraping intervals and targets to collect metrics from are defined.

```
Global:
  scrape_interval: 10s
scrape_configs:
  - job_name: Prometheus
    scrape_interval: 10s
    static_configs:
      - targets:
          - 'prometheus:9090'
    - job_name: node-exporter
    static_configs:
      - targets:
          - 'node-exporter:9100'
```

- datasources.yml file

This file defines the data source from which Grafana will fetch data, which is Prometheus. Note that it takes the URL values from the .env file defined earlier.

```
API version: 1
Data sources:
  - name: Prometheus
    type: Prometheus
    access: proxy # means requests will be made from the Grafana server instead of the user's browser
    url: https://${PROMETHEUS_CONTAINER}:${PROMETHEUS_PORT}
```

- dashboard.yml file

This file defines the dashboard configuration

```
API version: 1
providers:
  - name: "Our Dashboard Provider"
```

```
    orgID: 1
    type: file
    disableDeletion: false
    updateIntervalSeconds: 30
    allow updates: false
    options:
      path: /var/lib/grafana/dashboards
      foldersFromFileStructure: true
```

- docker-compose.yml file

This file will define the parameters required for deploying the containers that will compose our monitoring stack, including Prometheus, Grafana, and Node-Exporter.

```
# Define the custom network used by all services in this stack
networks:
  monitoring:
    driver: bridge
# Define the volumes for persistent data storage
volumes:
  prometheus_data:
    name: prometheus_data
  grafana_data:
    name: grafana_data
# Services
services:
  prometheus:
    image: prom/prometheus: latest
    container_name: ${PROMETHEUS_CONTAINER}
    restart: always
    env_file: .env
    Volumes:
    - ./config/prometheus.yml:/etc/prometheus/prometheus.yml
    expose:
    - ${PROMETHEUS_PORT}
    networks:
      - monitoring
    depends_on:
      - node-exporter
  Node-exporter:
```

image: prom/node-exporter: latest
container_name: ${NODE_EXPORTER_CONTAINER}
restart: always
env_file: .env
Volumes:
- /proc:/host/proc:ro
- /sys:/host/sys:ro
- /:/rootfs:ro
expose:
- ${NODE_EXPORTER_PORT}
networks:
  - monitoring
grafana:
image: grafana/grafana:latest
container_name: ${GRAFANA_CONTAINER}
restart: always
Ports:
- ${GRAFANA_PORT}:${GRAFANA_PORT}
env_file: .env
volumes:
- ./config/datasources.yml:/etc/grafana/provisioning/datasources/datasource.yml
- ./config/dashboard.yml:/etc/grafana/provisioning/datasources/dashboard.yml
- ./config/daashboard.json:/var/lib/grafana/dashboards/dashboard.json
networks:
  - monitoring
depends_on:
  - prometheus

The deployment of the stack was run using the following command, and the result that we got the three containers running as depicted in Figure 4:

 # docker compose up -d

Figure 4: Docker composition completed

Accessing Grafana on the default port 3000, Grafana UI, as in Figure 5.

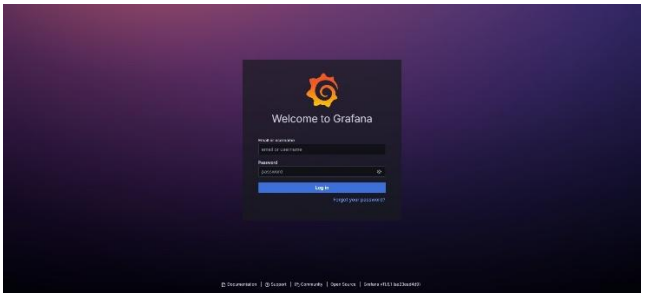

Figure 5: Grafana GUI

The rest of this paper will navigate through the capabilities of Grafana and how insightful dashboards can be created to maintain a real-time monitoring capability for infrastructure.

4.Results

This section describes the results obtained after deploying the monitoring stack composed of Prometheus, Grafana, and Node-Exporter. After accessing Grafana GUI, I needed to navigate to the Data Sources section to ensure that the Prometheus source had been added from the datasources.yml file mentioned above, and it ran successfully, as in Figure 6.

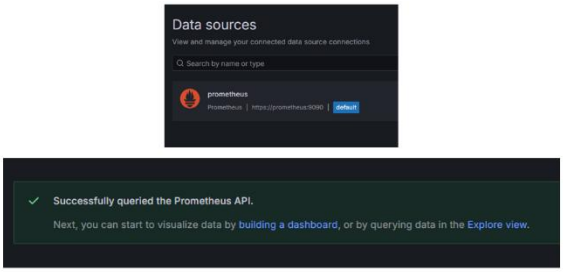

Figure 6: Prometheus data source added successfully

To get a quick and thorough insight about metrics collected, a node-exporter-full dashboard was imported, which provides an extensive overview of system performance using Prometheus-collected data to display key Linux metrics. It offers real-time tracking of CPU load, memory usage, disk activity, and network traffic. It enables comprehensive analysis of server health, facilitating optimization and troubleshooting, which provides a better view of resource utilization and identifying bottlenecks. The dashboards are illustrated in Figure 7 below:

Figure 7: Node Exporter Full dashboard highlights

There are many visualizations, represented by panels and categorized into rows, each providing insights about specific metrics. Below is the description for each one of the sections illustrated in Figure 7.

- Quick CPU/Mem/Disk

Provides a high-level summary of essential system resources, allowing rapid assessment of system uptime, CPU utilization, memory allocation, and disk activity. It serves as an essential checkpoint for identifying performance constraints or unexpected load behavior.

- Storage Disk

Offers a comprehensive breakdown of disk operations and helps in detecting early signs of disk efficiency and I/O latency trends, ensuring optimal data handling.

- Network Netstat

Deliver detailed insights into network communication, including active connections, transmission rates, and error occurrence for easily identifiable bottlenecks.

5 Discussions

Having a holistic view of infrastructure is of great importance in maintaining high performance and availability, especially for critical systems that might lead to catastrophic impact if any issues arise within. Hence, a reliable and insightful solution stack must be implemented to address such a necessity. The stack composed of Prometheus, Grafana, and Node Exporter forms a robust monitoring ecosystem.

Prometheus is used for collecting and storing time-series data for metrics, enabling precise and real-time performance analysis and anomaly detection in dynamic environments. Node Exporter serves as a vital data source, extracting detailed metrics from Linux-based infrastructures, including resources and their utilization. Grafana enhances this ecosystem by transforming raw metrics that have been queried using PromQL via Prometheus into intuitive graphs and offering advanced visualizations and dashboards.

Together, these tools facilitate large-scale observability, and leveraging their interoperability, intelligent monitoring solutions can be tailored to cope with demanding complex infrastructure environments, ensuring accuracy in diagnostic and proactive measures while maintaining efficiency in systems' operations.

While other monitoring solutions like Elastic stack [17] are also feasible in providing insights and dashboards, it is important to mention that the decision of which tools to adopt relies merely on the use cases. Table 3 depicts the comparison between our Prometheus, Node Exporter, and Grafana-implemented stack and the Elastic stack (ELK).

Table 3: Comparison of implemented stack with Elastic stack

Feature	Prometheus, Node Exporter and Grafana stack	Elastic Stack (ELK)
Primary Function	Collects, stores, and visualizes real-time metrics for performance monitoring and anomaly detection	Aggregates processes and indexes logs for comprehensive event analysis and security insights.
Data Handling	Collects structured time-series data optimized for numerical metrics	Indexes large volume logs, enabling text-based search and correlation

Data Sources	Acquires metrics from exporters (i.e., Node Exporter)	Ingests logs via Beats, Logstash, and direct API inputs
Scalability	Designed for high-performance, distributed metrics monitoring, ensuring minimal overhead	Optimized for large-scale log indexing and retrieval at the enterprise level
Use Case Focus	Ideal for real-time infrastructure monitoring, resource optimization, and predictive analysis	Tailored for log analysis, security monitoring, and compliance auditing in cyber security environments

6 Conclusion

In this paper, a reliable monitoring stack composed of open-source tools (i.e., Prometheus, Node Exporter, and Grafana) had been adopted to monitor a Linux server and gain insights about metrics including but not limited to CPU load, RAM consumption, disk utilization, …etc. For ease of deployment, docker had been leveraged, which provided a fast yet consistent service availability.

The results show that the implemented monitoring stack provides a holistic view of the monitored infrastructure by having intuitive dashboards composed of fine-tuned clean-view panels well organized in sections, which make it easy to observe and analyze anomalies. Noting that the dashboard had been imported for a quick run-to-go process.

It is recommended to conduct further studies to highlight some other features that even empower the Prometheus and Grafana Stack more and more, like implementing alerting capabilities. In addition, Loki also focuses on the logs aggregation process, which provides more perceptions.

Acknowledgement: Not Applicable.

Funding Statement: The author(s) received no specific funding for this study.

Conflicts of Interest: The authors declare no conflicts of interest to report regarding the present study.

References

[1] Leppänen, Tiia, “Data Visualization and Monitoring with Grafana and Prometheus,” *Information and Communications Technology- Bachelor’s Thesis*, 2021.

[2] Singh Abhishek, “A Data Visualization Tool- Grafana,” 2023.

[3] D. Ha, “Network monitoring use cases by industry, LogicMonitor,” Available at: https://www.logicmonitor.com/blog/network-monitoring-use-cases (Accessed: 25 April 2025).

[4] 5 challenges in monitoring complex IT infrastructure (no date) Netmonk. Available at: https://netmonk.id/blog/5-challenges-in-monitoring-complex-it-infrastructure (Accessed: 25 April 2025).

[5] A. Mehdi, "Unleashing the potential of Grafana: A comprehensive study on real-time monitoring and visualization," *2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT)* [Preprint], 2023.

[6] Prometheus Overview: Prometheus, Prometheus Blog. Available at: https://prometheus.io/docs/introduction/overview/ (Accessed: 26 April 2025).

[7] Prometheus Client libraries: Prometheus, Prometheus Blog. Available at: https://prometheus.io/docs/instrumenting/clientlibs/ (Accessed: 26 April 2025).

[8] Prometheus When to use the push gateway: Prometheus, Prometheus Blog. Available at: https://prometheus.io/docs/practices/pushing/ (Accessed: 26 April 2025).

[9] Prometheus Exporters and integrations: Prometheus, Prometheus Blog. Available at: https://prometheus.io/docs/instrumenting/exporters/ (Accessed: 26 April 2025).
[10] Prometheus Alertmanager: Prometheus, Prometheus Blog. Available at: https://prometheus.io/docs/alerting/latest/alertmanager/ (Accessed: 26 April 2025).
[11] What is Grafana, and what use cases of Grafana? DevOpsSchool.com. Available at: https://www.devopsschool.com/blog/what-is-grafana-and-use-cases-of-grafana/ (Accessed: 26 April 2025).
[12] P. B.C., H. Maddirala and M. S., "Implementing an effective infrastructure monitoring solution with Prometheus and Grafana," *International Journal of Computer Applications*, vol.186, no.38, pp. 7–15, 2024.
[13] S. Venkatramulu, Md. Sharfuddin Waseem, Arshiya Taneem, Sri Yashaswini Thoutam, Snigdha Apuri, and Nachiketh, "Research on SQL Injection Attacks using Word Embedding Techniques and Machine Learning," *Journal of Sensors, IoT & Health Sciences*, vol.2, no.1, pp.55-64, 2024.
[14] S H Sunil Kumar and C. Saravanan, "A Comprehensive Study on Data Visualization tool – Grafana," *International Journal of Emerging Technologies and Innovative Research*, vol.8, no. 5, pp: f908-f914, 2021.
[15] Application monitoring using Prometheus and Grafana. Available at: https://ijercse.com/article/6.8%20August%202024%20IJERCSE.pdf (Accessed: 02 May 2025).
[16] Y. Jani, "Unified Monitoring for microservices: Implementing prometheus and Grafana for Scalable Solutions," *Journal of Artificial Intelligence, Machine Learning and Data Science*, vol.2, no.1, pp. 848–852, 2024.
[17] Mohammed Daffalla Elradi, "Elastic Stack: A Reliable Tool for Highly-Demanding Data Insight," *Acta Scientific Computer Sciences*, vol.4, no.10, pp. 03-11, 2022.
[18] Sreedhar Bhukya and Shaik Khasim Saheb, "A Novel Approach for Analyzing Temporal Influence Dynamics in Social Networks," *Journal of Computer Allied Intelligence*, vol.2, no.2, pp.13-21, 2024.